

# Backend della dashboard meteo

## Node.js + Express + Open-Meteo + Docker + Render

**Scopo:** Portare il backend da cartella vuota a servizio pubblico funzionante. Il documento non fornisce il codice completo delle API meteo: indica con precisione cosa realizzare, dove inserirlo, quale documentazione leggere e come verificare ogni passaggio.

### 1. Risultato finale atteso

Il backend deve esporre tre funzioni pubbliche: controllo dello stato, ricerca di un luogo e previsione meteo di cinque giorni. Deve ricevere richieste dal frontend React, interrogare Open-Meteo, trasformare i dati e restituire risposte JSON stabili.

```
GET /api/health
GET /api/weather/locations?q=Roma
GET /api/weather/forecast?lat=41.89&lon=12.51
```

- Nessuna registrazione a Open-Meteo.
- Nessuna API key.
- Nessun database nella prima versione.
- Docker usato per rendere l'ambiente ripetibile.
- Render usato per rendere il backend disponibile online.

### 2. Architettura e flusso delle richieste

```
Browser / React
  | richiesta HTTP
  v
Backend Express
  | validazione input
  | chiamata esterna
  v
Open-Meteo
  | risposta grezza
  v
Normalizzazione backend
  | JSON del progetto
  v
Frontend
```

La separazione è importante: routes riceve la richiesta, services comunica con Open-Meteo, utils trasforma i dati, app configura Express, server avvia il processo.

### 3. Documentazione ufficiale da usare

**Open-Meteo Forecast API** - <https://open-meteo.com/en/docs>

Per scegliere parametri daily, forecast\_days, timezone e unità di misura.

**Open-Meteo Geocoding API** - <https://open-meteo.com/en/docs/geocoding-api>

Per capire name, count, language, format e il formato dei risultati.

**Express - installazione** - <https://expressjs.com/en/starter/installing.html>

Per verificare versione Node richiesta e installazione Express.

**Express - routing** - <https://expressjs.com/en/starter/basic-routing.html>

Per comprendere metodi HTTP, path, req e res.

**Express - error handling** - <https://expressjs.com/en/guide/error-handling.html>

Per implementare middleware centralizzato degli errori.

**Docker per Node.js** - <https://docs.docker.com/guides/nodejs/containerize/>

Per build, immagine, .dockerignore e avvio del container.

**Docker Compose** - <https://docs.docker.com/compose/>

Per eseguire più servizi quando verrà aggiunto il frontend.

**Render Express deploy** - <https://render.com/docs/deploy-node-express-app>

Per configurare repository, build command, start command e variabili.

**Node.js download** - <https://nodejs.org/en/download>

Per installare una versione LTS e verificare node/npm.

**Metodo di studio:** Non leggere tutta la documentazione. Apri la pagina collegata quando arrivi alla fase corrispondente e verifica soltanto i parametri o il concetto che devi usare.

### 4. Prerequisiti e verifica iniziale

| Strumento      | Perché serve          | Verifica                      | Risultato minimo                |
|----------------|-----------------------|-------------------------------|---------------------------------|
| Node.js LTS    | Runtime backend e npm | <code>node --version</code>   | Una versione LTS installata     |
| npm            | Gestione dipendenze   | <code>npm --version</code>    | Comando disponibile             |
| Git            | Versionamento         | <code>git --version</code>    | Comando disponibile             |
| Docker Desktop | Container locale      | <code>docker --version</code> | Docker Engine attivo            |
| VS Code        | Editor                | Aprire la cartella            | Terminale integrato funzionante |

```
node --version
npm --version
git --version
docker --version
```

| Strumento              | Perché serve | Verifica | Risultato minimo |
|------------------------|--------------|----------|------------------|
| docker compose version |              |          |                  |
| docker run hello-world |              |          |                  |

## 5. Creazione del progetto

1. Creare una cartella weather-backend.
2. Aprire la cartella in VS Code.
3. Inizializzare npm.
4. Installare Express, cors, helmet e dotenv.
5. Installare nodemon come dipendenza di sviluppo.
6. Configurare package.json con type module, script dev e script start.

```
mkdir weather-backend
cd weather-backend
npm init -y
npm install express cors helmet dotenv
npm install --save-dev nodemon
```

**Checkpoint:** Il file package.json e package-lock.json devono esistere; node\_modules non deve essere caricato su Git.

## 6. Struttura precisa dei file

```
weather-backend/
|-- src/
|   |-- routes/
|   |   `-- weather.routes.js
|   |-- services/
|   |   `-- openMeteo.service.js
|   |-- utils/
|   |   `-- forecast.js
|   |-- app.js
|   `-- server.js
|-- tests/
|-- .env
|-- .env.example
|-- .gitignore
|-- .dockerignore
|-- Dockerfile
|-- package.json
`-- package-lock.json
```

**src/server.js** - Legge le variabili e mette Express in ascolto.

**src/app.js** - Registra middleware, health API, route meteo, 404 e error handler.

**src/routes/weather.routes.js** - Contiene il contratto HTTP delle API locations e forecast.

**src/services/openMeteo.service.js** - Conosce gli URL esterni e traduce gli errori di rete.

**src/utils/forecast.js** - Converte gli array di Open-Meteo in cinque oggetti giornalieri.

**tests/** - Conterrà test delle route e delle funzioni di trasformazione.

**.env** - Configurazione locale esclusa da Git.

**.env.example** - Elenco pubblico delle variabili richieste.

**Dockerfile** - Ricetta per costruire il container.

## 7. Configurazione dell'ambiente

Creare .env con porta, ambiente e origine del frontend. Open-Meteo non richiede segreti.

```
PORT=8080
NODE_ENV=development
FRONTEND_URL=http://localhost:5173
```

Creare .env.example con gli stessi nomi. Creare .gitignore:

```
node_modules
.env
coverage
npm-debug.log
.DS_Store
```

## 8. Prima API: health check

Questa è l'unica API per cui il documento include il codice base. Serve a verificare che Express, CORS, Helmet, la porta e il routing funzionino prima di aggiungere Open-Meteo.

### src/app.js

```
import express from "express";
import cors from "cors";
import helmet from "helmet";

const app = express();

app.use(helmet());
app.use(express.json());
app.use(cors({ origin: process.env.FRONTEND_URL }));

app.get("/api/health", (req, res) => {
  res.status(200).json({ status: "ok", service: "weather-backend" });
});

export default app;
```

### src/server.js

```
import "dotenv/config";
import app from "./app.js";

const port = Number(process.env.PORT) || 8080;

app.listen(port, "0.0.0.0", () => {
  console.log(`Backend avviato sulla porta ${port}`);
});
```

```
});
```

Avviare con `npm run dev` e verificare nel browser o con `curl`:

```
curl http://localhost:8080/api/health
```

**Risultato atteso:** HTTP 200 con JSON contenente `status=ok` e `service=weather-backend`. Se non funziona, non procedere.

## 9. Progettare il contratto delle API prima del codice

Scrivere nel README gli endpoint, i parametri, le risposte e gli errori. Questo evita che frontend e backend interpretino i dati in modo diverso.

| Endpoint                   | Input                 | Output                        | Errori principali        |
|----------------------------|-----------------------|-------------------------------|--------------------------|
| GET /api/health            | Nessuno               | Stato del servizio            | 500 solo in caso anomalo |
| GET /api/weather/locations | q, almeno 2 caratteri | Elenco di luoghi normalizzati | 400 input, 502 provider  |
| GET /api/weather/forecast  | lat e lon numerici    | Location + 5 giorni           | 400 input, 502 provider  |

## 10. Implementare la ricerca dei luoghi

### 10.1 Leggere la documentazione

7. Aprire la Geocoding API ufficiale.
8. Individuare endpoint `/v1/search`.
9. Leggere i parametri `name`, `count`, `language` e `format`.
10. Osservare i campi `results`, `name`, `latitude`, `longitude`, `country`, `country_code`, `admin1` e `timezone`.
11. Provare una richiesta dalla pagina documentale prima di scrivere il servizio.

### 10.2 Creare il servizio

- Definire l'URL base in un solo punto.
- Creare una funzione che riceve query già pulita.
- Costruire i parametri con `URLSearchParams`.
- Impostare `count=5`, `language=it` e `format=json`.
- Usare `fetch` di `Node.js`.
- Controllare `response.ok`.
- Se `results` manca, usare una lista vuota.
- Mappare soltanto i campi del contratto interno.

### 10.3 Creare la route

- Leggere `req.query.q`.
- Convertire a stringa e rimuovere spazi iniziali/finali.
- Rifiutare input inferiori a due caratteri con HTTP 400.
- Chiamare il servizio.

- Restituire { locations: [...] }.
- Passare gli errori al middleware centralizzato.

## 10.4 Verificare

- Roma deve produrre risultati italiani.
- Springfield deve produrre località omonime.
- Una stringa casuale può restituire lista vuota.
- q mancante e q di un carattere devono restituire 400.

# 11. Implementare il forecast di cinque giorni

## 11.1 Leggere la documentazione

12. Aprire Weather Forecast API.
13. Inserire coordinate di prova nella console della documentazione.
14. Selezionare Daily Weather Variables.
15. Aggiungere weather\_code, temperature\_2m\_max, temperature\_2m\_min, precipitation\_probability\_max e wind\_speed\_10m\_max.
16. Impostare timezone=auto e forecast\_days=5.
17. Osservare che daily contiene array paralleli con gli stessi indici.

## 11.2 Validare le coordinate

- Convertire lat e lon in Number.
- Rifiutare NaN e valori infiniti.
- Latitudine valida: -90..90.
- Longitudine valida: -180..180.
- Restituire HTTP 400 con messaggi distinti.

## 11.3 Creare il servizio forecast

- Ricevere coordinate validate.
- Costruire i parametri giornalieri definiti nel contratto.
- Impostare timezone=auto.
- Impostare forecast\_days=5.
- Controllare response.ok e distinguere errore del provider da errore interno.
- Restituire il JSON grezzo alla funzione di normalizzazione.

## 11.4 Normalizzare la risposta

Per ogni indice dell'array daily.time creare un singolo oggetto. Verificare che gli array richiesti esistano e abbiano lunghezza coerente. Il risultato interno deve avere esattamente cinque elementi, salvo errore esplicito del provider.

- date
- weatherCode
- description italiana
- temperatureMin
- temperatureMax
- precipitationProbability

- windSpeed

La descrizione italiana deriva dal codice WMO. Conservare la tabella codice-descrizione in forecast.js e prevedere un valore di fallback.

## 12. Gestione degli errori e robustezza

- Aggiungere una route finale 404 dopo tutte le route valide.
- Aggiungere middleware error handler con quattro parametri.
- Non inviare stack trace al browser in produzione.
- Registrare messaggio e causa nei log.
- Usare HTTP 502 quando Open-Meteo non risponde correttamente.
- Prevedere timeout della chiamata esterna tramite AbortController.
- Restituire sempre JSON anche in caso di errore.
- Non lasciare promise senza await o catch.

## 13. Test necessari

Prima fare test manuali; successivamente aggiungere test automatici con un framework come Vitest o Jest e Supertest.

| Area      | Caso                    | Risultato           |
|-----------|-------------------------|---------------------|
| Health    | GET /api/health         | 200 e JSON corretto |
| Locations | Roma                    | Lista non vuota     |
| Locations | q mancante              | 400                 |
| Locations | Nessun risultato        | 200 con lista vuota |
| Forecast  | Coordinate Roma         | 5 giorni            |
| Forecast  | lat=abc                 | 400                 |
| Forecast  | lat=100                 | 400                 |
| Provider  | Errore esterno simulato | 502 controllato     |
| 404       | Endpoint casuale        | 404 JSON            |

## 14. Docker: configurazione completa

Creare il Dockerfile solo dopo che /api/health funziona fuori da Docker.

### Dockerfile

```
FROM node:22-alpine
```

```
WORKDIR /app
```

```
COPY package*.json ./
RUN npm ci --omit=dev
```

```
COPY src ./src
```

```
ENV NODE_ENV=production
EXPOSE 8080
```

```
CMD ["npm", "start"]
```

## **.dockerignore**

```
node_modules
.env
coverage
.git
npm-debug.log
```

## **Build e avvio**

```
docker build -t weather-backend .
docker run --rm -p 8080:8080 --env-file .env weather-backend
```

- FROM fornisce Node.js e Linux minimale.
- WORKDIR evita percorsi dispersi.
- COPY package prima del sorgente migliora la cache.
- npm ci usa package-lock per build ripetibili.
- --omit=dev esclude nodemon dal container di produzione.
- 0.0.0.0 nel server rende il processo raggiungibile dal mapping della porta.
- --env-file passa configurazione senza incorporarla nell'immagine.

**Test Docker:** Con il container attivo, aprire nuovamente /api/health e poi entrambe le API meteo. Il comportamento deve essere identico all'esecuzione con npm run dev.

## **15. Collegamento al frontend**

- Confermare che FRONTEND\_URL sia http://localhost:5173 in sviluppo.
- Comunicare al frontend il base URL http://localhost:8080/api.
- Non cambiare i nomi dei campi senza aggiornare il contratto condiviso.
- Provare prima locations dal frontend, poi forecast.
- Aprire DevTools Network per controllare status e JSON.
- Se compare errore CORS, verificare origine esatta, protocollo e porta.

## **16. GitHub e branch**

```
git init
git add .
git status
git commit -m "Create backend foundation"
```

- feature/health-api
- feature/location-search
- feature/weather-forecast



- feature/backend-errors
- feature/backend-docker

Prima di ogni push controllare che .env e node\_modules non compaiano in git status.

## 17. Deploy del backend su Render

18. Caricare il repository su GitHub.
19. Aprire Render Dashboard e creare New Web Service.
20. Collegare il repository.
21. Se il repository contiene frontend e backend, impostare Root Directory su backend.
22. Usare runtime Node per il percorso più semplice.
23. Build Command: npm install oppure npm ci.
24. Start Command: npm start.
25. Aggiungere FRONTEND\_URL con il dominio Vercel definitivo.
26. Verificare che il server usi process.env.PORT.
27. Eseguire il deploy e aprire /api/health sull'URL pubblico.
28. Provare locations e forecast dal browser o da Postman.

**Nota:** Il piano gratuito può sospendere il servizio inattivo; la prima richiesta dopo un periodo di inattività può essere lenta.

## 18. Problemi frequenti

**Cannot use import statement** - Manca type=module o sintassi dei moduli incoerente.

**EADDRINUSE** - La porta 8080 è già occupata.

**CORS blocked** - FRONTEND\_URL non corrisponde esattamente all'origine del browser.

**fetch is not defined** - Versione Node troppo vecchia; usare una LTS moderna.

**Container parte e si chiude** - CMD o script start errato.

**Render health non risponde** - Server non ascolta su process.env.PORT o su 0.0.0.0.

**Forecast vuoto** - Parametri daily errati o coordinate non valide.

**Descrizioni mancanti** - Codice WMO non presente nella mappa; aggiungere fallback.

## 19. Definition of Done backend

- ☐ Installazione ripetibile con npm ci.
- ☐ /api/health risponde 200 in locale e Docker.
- ☐ Ricerca luogo gestisce risultati, omonimi e lista vuota.
- ☐ Forecast restituisce cinque giorni normalizzati.

- ☐ Input invalidi restituiscono 400.
- ☐ Errori provider restituiscono risposta controllata.
- ☐ Nessun segreto o .env nel repository.
- ☐ Immagine Docker costruibile.
- ☐ Backend pubblico su Render.
- ☐ Frontend autorizzato tramite CORS.
- ☐ README contiene setup, endpoint e test manuali.